

TD 9 injection sql

Matisse kra

bts sio

Sommaire

1) Problématique	1
2) Etapes	1
3) Conclusion	3

1) Problématique

Nous souhaitons suivre une formation sur le thème de la sécurisation des applications Web. Durant cette formation, nous allons jouer le rôle du hacker et celui du développeur. Nous nous aiderons des vidéos pour réaliser les différentes étapes du projet.

2) Etapes

Etape 1 : Configuration de l'environnement

- Il est nécessaire d'avoir VirtualBox sur son PC avec un minimum de 6 GO de mémoire vive. Nous avons procédé à l'installation des 4 machines virtuelles requises. Une fois les VMs importées, elles s'ajoutent automatiquement à l'interface de VirtualBox.
- Nous avons suivi la vidéo de configuration présente sur le bureau de la VM Ubuntu légitime. Les noms des cartes réseau ont été vérifiés (srv-in en réseau interne). Pour la VM client légitime, les paramètres IP ont été passés de automatique à manuel pour fixer l'adresse réseau.

carte reseaux serveur ubuntu



carte reseaux VM client légitime

Réseau

Adapter 1

☒ Activer l'interface réseau

Attached to Réseau interne

Name lan-in

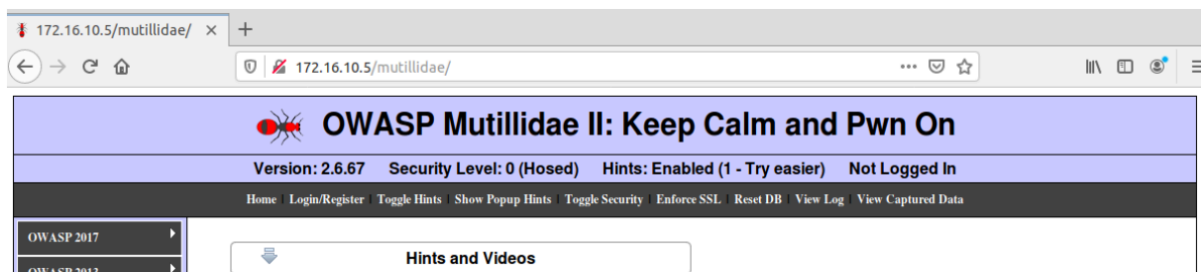
paramètres IPV4

Adresse	Masque de réseau	Passerelle
192.168.50.10	24	192.168.50.254

vérification de la connexion avec la commande ip a s

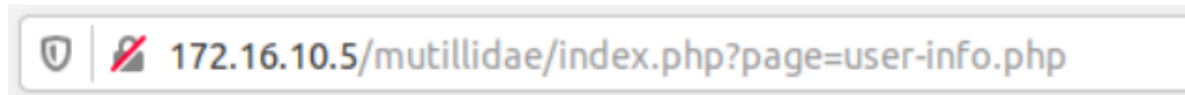
```
prof@prof:~$ ip a s
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d8:63:73 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.10/24 brd 192.168.50.255 scope global noprefixroute enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::af3a:837c:9a40:62fa/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

test connexion au serveur sur firefox



Etape 2 : Simulation d'attaque SQL (Niveau 0)

Dans cette phase, nous avons configuré le réseau de la VM HACKER pour accéder au site cible via le serveur donner dans la video.

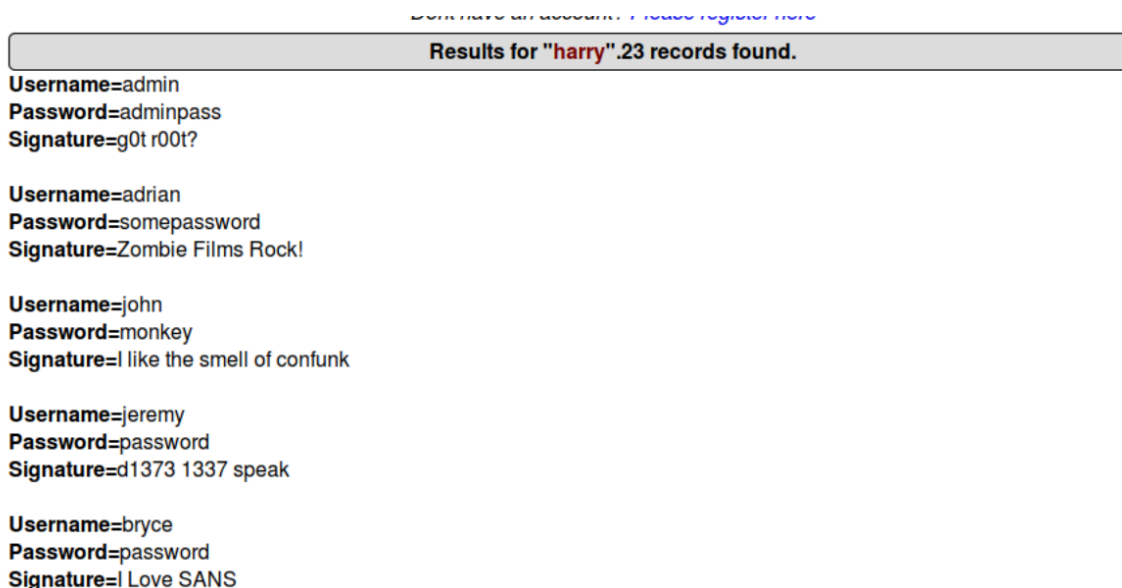


Nous avons accédé à la page user-info pour tester une injection SQL classique sur le site Mutillidae.

en utilisant un identifiant spécifique, l'injection SQL permet d'accéder aux informations confidentielles de la base de données. Cela démontre que lorsque le site est au niveau de sécurité 0, il est totalement vulnérable.

Sur la page d'authentification, saisir les informations suivantes :
login : harry
mot de passe : 'or 'a' = 'a

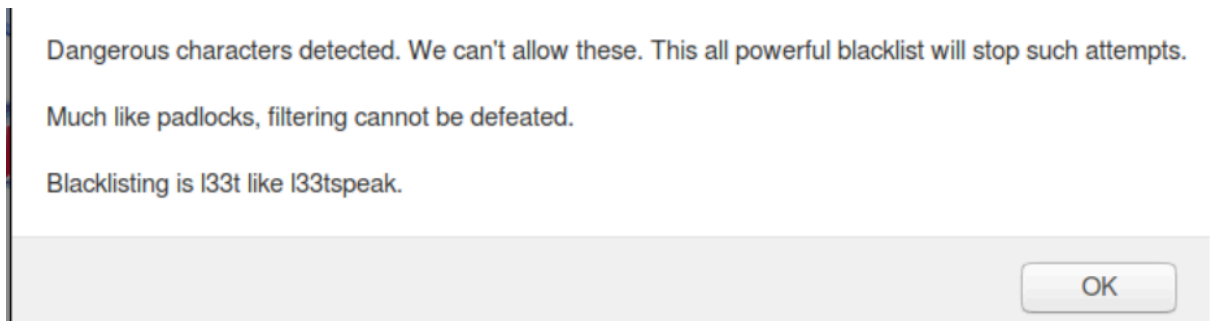
On tombe sur le résultat suivant :



Etape 3 : Sécurisation et analyse de code (Niveau 5)

Désormais, nous augmentons le niveau de sécurité à 5 via l'option "Toggle Security".

- **Protection active** : En réessayant l'injection, une alerte s'affiche. Le système utilise désormais une liste noire et vérifie la longueur des données saisies.



- **Analyse de code** : La comparaison des sources montre l'activation d'un script de filtrage des caractères spéciaux avant l'envoi du formulaire.

la version sécurisée de niveau 5

```
<script type="text/javascript">  
    var lValidateInput = "TRUE"
```

la version D'avant :

```
<script type="text/javascript">  
    var lValidateInput = "FALSE"
```

La version sécurisée met en œuvre un script de filtrage des caractères spéciaux préalablement à l'envoi du formulaire. On peut donc en déduire qu'en matière de bonnes pratiques, il est absolument indispensable d'intégrer un script de filtrage des entrées utilisateur au sein de son code afin de prémunir son site contre les injections SQL.

3) Conclusion

En conclusion, grâce à cette formation, j'ai compris le fonctionnement technique des injections SQL. Cette expérience a mis en évidence l'importance cruciale d'implémenter des scripts de filtrage et de validation des entrées utilisateur pour protéger efficacement une application web contre les cyberattaques.